Check-In Protocol sends a sessionless message (Check-In message) that relies on a key that has been given to the server during the registration of the client.

4.22.1. Requirements

4.22.1.1. Server Requirements

A Server use case of the Check-In Protocol SHALL implement these requirements.

- A persistent data store for Keys.
- An implementation of the Check-In Counter.

Persistent Data Store

For the Check-In Protocol to be effective, the use case of the protocol SHALL provide a means to store and persist client registration information. The minimum information that SHALL be stored and persisted is the key to use during the encryption process of the Check-In message.

4.22.1.2. Client Requirements

The server does not store the starting value of the Check-In Counter for a given key. Therefore, it is the client's responsibility to do so when the client registers with the server. A Client use case of the Check-In Protocol SHALL implement a means to store sets containing a key, the starting value of Check-In Counter and the last known valid offset from the starting value of the Check-In Counter. A Key that has been used to register for Check-In messages SHALL always be associated with a starting Check-In Counter value and the last known valid offset from the starting Check-In Counter value.

During the decryption process, the client uses the key, its associated starting Check-In Counter value and offset to decrypt and authenticate a received Check-In message.

4.22.2. Message Content

Table 55. Check-In Payload

Field	Type	Description
nonce	byte[CRYPTO_AEAD_NONCE_LENGTH_BYTES]	First plaintext field
Check-In Counter	uint32	Encrypted field - Counter used to generate the nonce
Application Data	byte[]	Encrypted field - Application Data that can be added to the Check-In message
MIC	byte[CRYPTO_AEAD_MIC_LENGTH_BYTES]	Message integrity Check

